

ENGSE203 — สอบกลางภาค ส่วน B • ปฏิบัติ (Coding) • Sec 1 / ชุด A

40 คะแนน • เวลา 180 นาที (3 ชม.) • ใช้ AI/เอกสาร/เว็บได้

ชื่อ-รหัส: _____ เครื่องที่: _____

กติกา — อ่านก่อนเริ่ม

ทำได้	ห้ามทำ
✓ ใช้ AI (ChatGPT, Copilot ฯลฯ), React docs, MDN, Google	✗ คูยกับเพื่อน / ส่งไฟล์หากัน
✓ ใช้ DevTools ทุกอย่าง	✗ แก่ไฟล์ในโฟลเดอร์ services/ (เขียนว่า “ให้มาแล้ว — ห้ามแก้”)

เงื่อนไขบังคับ 2 อย่าง

1. **commit** ทุกครั้งที่ผ่าน **checkpoint** ด้วยข้อความสื่อความหมาย — ผู้สอนดูประวัติ commit
2. กรอก **AI_USAGE.md** ระหว่างทำ — ถามอะไร ใช้คำตอบส่วนไหน แก่เองตรงไหน
ทุกงานจะถูกสุ่มถามใน **Oral Defense** ถ้าอธิบายโค้ดที่ส่งไม่ได้ คะแนนส่วนนั้นถูกทบทวน
• ใช้ AI เพื่อเข้าใจ ไม่ใช่เพื่อลอก

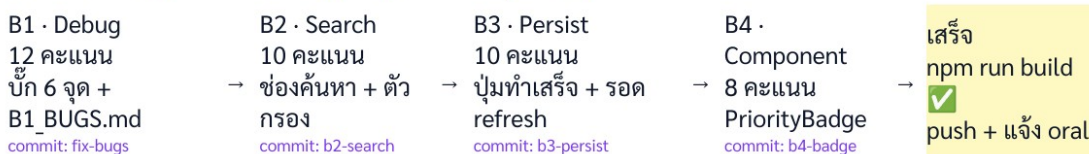
วิธีทำงาน — เดินทีละ checkpoint

วงจรทำงานต่อ 1 checkpoint — ทำซ้ำจนครบทุกงาน



ถ้าข้อ 5 ไม่ตรง กลับไปข้อ 2 (ทำนายใหม่ว่าเพราะอะไร) — อย่าเดาสุ่มหรือก๊อป AI ทั้งก่อน

แผนที่ checkpoint ทั้งภาคปฏิบัติ (commit ทุกกล่อง)



workflow

หลักง่าย ๆ: ก่อนแก้ไข ทำนายผล ก่อนว่าจะเกิดอะไร → แก้ → npm run dev แล้ว สังเกตจริง → ตรงกับ “ตัวอย่าง output” ในโจทย์ไหม → ผ่านแล้ว **commit** · ถ้าไม่ตรง กลับไปทำนายใหม่ อย่าเดาสุ่ม

เตรียมตัว

```
cd engse203-midterm-secA
```

```
npm ci
```

```
npm run dev # เปิด http://localhost:5173
```

เปิด AI_USAGE.md และ B1_BUGS.md ค้างไว้ · แอปคือ **Campus Service Request** เวอร์ชันเดียวกับ LAB

B1 · Debug — หา/แก้บั๊ก 6 จุด (12 คะแนน · ~45 นาที)

แอปนี้ **build** ผ่าน (ไม่มี syntax error) แต่พฤติกรรมผิด 6 จุด · แก้ทีละจุด แล้วบันทึกใน B1_BUGS.md (ไฟล์/บรรทัด/สาเหตุ/วิธีแก้) · **ข้อละ 2 คะแนน** (หาเจอ 0.5 · แก้ถูก 1 · อธิบายได้ 0.5)

แนะนำ: commit หลังแก้ครบ หรือ commit ทุก 2–3 บั๊ก — `git commit -m "B1: fix bug 1-3"`

CP-B1 · อาการที่ผู้ใช้รายงาน (ไล่ทีละข้อ)

บั๊ก 1 — เปิดหน้า Dashboard แล้วเปิด Console เห็น **คำเตือนสีเหลืองเรื่องรายการ (list)**

บั๊ก 2 — ตัวเลข “**รอดำเนินการ**” ในแผงสรุป ไม่ตรงกับจำนวนที่เห็นจริง

บั๊ก 3 — กดตัวกรอง “**รอดำเนินการ**” แล้วได้รายการที่ **ไม่ใช่** รอดำเนินการ

บั๊ก 4 — เปิด `#/requests/REQ-001` แล้วแก้ URL เป็น REQ-002 — **ข้อมูลบนหน้าจอไม่เปลี่ยน**

บั๊ก 5 — กดปุ่ม “**ลบ**” ที่การ์ด — รายการยังอยู่บนจอ ต้อง **รีเฟรชถึงหาย** (แต่รีเฟรชแล้วหายจริง = ข้อมูลถูกลบแล้ว)

บั๊ก 6 — กดปุ่ม “**Reset Demo Data**” แล้ว หน้าพัง/ว่างเปล่า

✓ ตัวอย่าง output ที่ถูกต้อง (ใช้ยืนยันว่าแก้สำเร็จ)

- **บั๊ก 1 แก้แล้ว:** Console **ไม่มี** คำเตือน Each child in a list should have a unique "key" prop
- **บั๊ก 2 แก้แล้ว:** แผงสรุปแสดง **ทั้งหมด 5 · รอดำเนินการ 3 · กำลังดำเนินการ 1 · เสร็จสิ้น 1** (ตามข้อมูลตัวอย่าง) — ก่อนแก้จะแสดงรอดำเนินการ 1 ทั้งที่มีการ์ด pending 3 ใบ
- **บั๊ก 3 แก้แล้ว:** กด “รอดำเนินการ” เหลือเฉพาะการ์ดที่สถานะ pending เท่านั้น
- **บั๊ก 4 แก้แล้ว:** เปลี่ยน URL เป็น REQ-002 หน้าจอเปลี่ยนเป็นข้อมูลของ REQ-002 ทันที
- **บั๊ก 5 แก้แล้ว:** กดลบแล้วการ์ด หายทันที **ไม่ต้องรีเฟรช**

- **บ๊ัก 6 แก้แล้ว:** กด Reset แล้วข้อมูลกลับเป็นชุดตัวอย่าง 5 รายการ หน้าไม่พัง
- checkpoint ผ่านเมื่อ:** ทิ้ง 6 อาการหายครบ + B1_BUGS.md กรอกครบ → **commit B1:**
fix all 6 bugs
-

B2 · Search — เพิ่มช่องค้นหาใน Dashboard (10 คะแนน · ~35 นาที)

เพิ่มช่องค้นหาเหนือรายการคำร้อง ทำเป็น **4 checkpoint** ย่อย commit ทุก checkpoint

CP-B2.1 — ช่องค้นหาแสดงและรับ input ได้ (3 คะแนน)

เพิ่ม `<input>` เหนือรายการ placeholder "ค้นหาจากผู้แจ้งหรือรายละเอียด" เก็บค่าด้วย `useState`

✅ **output:** พิมพ์แล้วตัวอักษรปรากฏในช่อง (ยังไม่ต้องกรอง) → **commit B2.1:** add search input

CP-B2.2 — กรองตามผู้แจ้ง/รายละเอียด (3 คะแนน)

กรองรายการที่ `requesterName` หรือ `details` มีข้อความที่พิมพ์ (ไม่สนตัวพิมพ์เล็กใหญ่)

✅ **output:** พิมพ์ ห้อง → เหลือ 2 การ์ด คือ **REQ-002** และ **REQ-003** → **commit B2.2:**
filter by name/details

CP-B2.3 — ทำงานร่วมกับตัวกรองสถานะ (2 คะแนน)

ค้นหาต้องกรองซ้อนกับตัวกรองสถานะเดิม เช่น กรอง `completed` + พิมพ์ ห้อง → เหลือเฉพาะที่ตรงทั้งสอง (REQ-003)

✅ **output:** สลับตัวกรอง + พิมพ์ค้นหา ได้ผลตรงทั้งสองเงื่อนไข → **commit B2.3:** combine with status filter

CP-B2.4 — ไม่พบ + แฉงสรุปไม่เปลี่ยน (2 คะแนน)

ถ้าค้นแล้วไม่เจอ แสดงข้อความ “ไม่พบคำร้องที่ตรงกับการค้นหา” · แฉงสรุปยังนับจากข้อมูลทั้งหมด ไม่เปลี่ยนตามการค้นหา

✅ **output:** พิมพ์ `zzz` → ขึ้นข้อความไม่พบ แต่แฉงสรุปยังเป็น **5/3/1/1** → **commit B2.4:**
empty message + summary from all

⚠️ **จุดที่ AI มักพลาด (และ oral ถามแน):** แฉงสรุปต้องคำนวณจาก requests (ทั้งหมด) ไม่ใช่จากรายการที่กรอง/ค้นหาแล้ว

B3 · Persist — ปุ่ม “ทำเสร็จ” ที่รอด refresh (10 คะแนน · ~50 นาที)

ในไฟล์ `services/requestService.js` มีฟังก์ชัน `updateRequestStatus(requestId, nextStatus)` ให้มาแล้ว (อ่าน → แก่แบบ immutable → เขียน localStorage → คำนวณใหม่) · งานของคุณคือ ต่อ UI เข้ากับมัน และทำให้หน้าจอสะท้อนผล + รอด refresh

CP-B3.1 — เพิ่มปุ่ม “ทำเสร็จ” ที่การ์ด (3 คะแนน)

ในการ์ดคำร้องที่สถานะ **ยังไม่** completed ให้มีปุ่ม “ทำเสร็จ” (ส่ง callback ขึ้นไปที่หน้า Dashboard)

✓ **output:** การ์ดที่ยังไม่ completed (REQ-001, REQ-002, REQ-004, REQ-005) มีปุ่ม “ทำเสร็จ” · REQ-003 (completed) ไม่มี → **commit B3.1:** add mark-done button

CP-B3.2 — กดแล้วสถานะเปลี่ยน + summary อัปเดต (4 คะแนน)

เขียน handler เรียก `updateRequestStatus(id, 'completed')` แล้ว `setRequests(...)` ด้วยชุดที่ได้กลับมา

✓ **output:** กด “ทำเสร็จ” ที่ REQ-001 → badge เปลี่ยนเป็น completed, ปุ่มหายไป, แผงสรุป เปลี่ยนเป็น รอดดำเนินการ 2 · เสร็จสิ้น 2 ทันที → **commit B3.2:** update status + refresh

CP-B3.3 — รอด refresh (persist จริง) (3 คะแนน)

กด F5 รีเฟรชหน้า — สถานะที่เพิ่งเปลี่ยนต้อง **ยังอยู่** (เพราะเขียนลง localStorage ผ่าน service แล้ว)

✓ **output:** หลังรีเฟรช REQ-001 ยังเป็น completed, สรุปยังเป็น 0/.../2 → **commit B3.3:** verify persistence

ตรวจใน DevTools → Application → Local Storage จะเห็นคีย์ `engse203-campus-requests-v1` เก็บค่าใหม่

B4 · From Scratch — สร้าง PriorityBadge (8 คะแนน · ~30 นาที)

สร้าง component ใหม่ `src/components/PriorityBadge.jsx` แล้วนำไปใช้แทนการแสดงผล priority ดิบใน RequestCard

CP-B4.1 — สร้างไฟล์ + แสดง 2 กรณีหลัก (4 คะแนน)

- รับ prop `priority`
- `'urgent'` → แสดง “เร่งด่วน” `className="priority-urgent"`
- `'normal'` → แสดง “ปกติ” `className="priority-normal"`

✓ **output:** REQ-001 (urgent) → “เร่งด่วน” · REQ-002/003 (normal) → “ปกติ” → **commit B4.1:** PriorityBadge base

CP-B4.2 — จัดการ edge case (2 คะแนน)

ถ้า priority เป็นค่าอื่นหรือไม่มี → แสดง “ไม่ระบุ” className="priority-unknown"

วิธีทดสอบ: เพิ่มบรรทัดชั่วคราวใน RequestCard.jsx เพื่อดูผล แล้วลบออกเมื่อยืนยันแล้ว

```
<PriorityBadge priority="high" />  {/* ทดสอบชั่วคราว — ต้องแสดง "ไม่ระบุ" */}
```

✓ **output:** ค่าที่ไม่ใช่ urgent/normal → แสดง “ไม่ระบุ” (ไม่ใช่ว่างเปล่าหรือหน้าพัง) → commit B4.2: unknown case

⚠ อย่าทดสอบด้วยการแก้ initialRequests.json — ระบบตรวจสอบข้อมูลจะปฏิเสธค่าที่ไม่ใช่ normal/urgent แล้วขึ้นหน้า error แทน

CP-B4.3 — นำไปใช้ใน RequestCard (2 คะแนน)

แทน {request.priority} ดิบ ด้วย <PriorityBadge priority={request.priority} /> (ดู TODO B4 ในไฟล์)

✓ **output:** ทุกการ์ดแสดง badge สีตามความเร่งด่วน → commit B4.3: use badge in card

⚠ **oral ถามแน:** “ทำไมถึงคิดถึงกรณีค่าอื่น” — คนที่ใส่ edge case เองคือคนที่เข้าใจ

ก่อนหมดเวลา — เช็คลิสต์

- ☐ B1 แก้ครบ 6 จุด + B1_BUGS.md กรอกครบ
- ☐ B2 ค้นหาทำงานครบ 4 checkpoint
- ☐ B3 ปุ่มทำเสร็จ persist + รอด refresh
- ☐ B4 PriorityBadge + ใช้ใน RequestCard
- ☐ npm run build ผ่าน
- ☐ AI_USAGE.md กรอกครบ
- ☐ commit + push ครั้งสุดท้าย
- ☐ แจ้งผู้สอนว่าพร้อม oral

สรุปคะแนนส่วน B

งาน	คะแนน	CLO	สัปดาห์
B1 Debug (6 จุด)	12	4,5	W4 + W5A + W5B
B2 Search	10	4,5	W5A (read)
B3 Persist	10	4,5	W5B (write)
B4 PriorityBadge	8	4	W4

งาน	คะแนน CLO	สัปดาห์
รวม	40	